

Datenbanksysteme – Abfrage von Daten

Zbigniew Jerzak

Berliner Hochschule für Technik
Medieninformatik Bachelor
Wintersemester 2025
Zug 2

14. November 2025

Roter Faden

LVE

Recap

Subqueries

Single-Row-Subquery

Ausland/LVE

Multi-Row-SubQ

Correlated Subquery

CTE

Views

Satz-Ops

UNION

INTERSECT

EXCEPT

Fazit

Quiz

Welche Zahl kommt als Nächste in der Reihe:

1, 11, 21, 1211, 111221, 312211, ...

Lehrveranstaltungsevaluation

- ▶ Dauer: 10-15 Minuten
- ▶ Umfang: alle bisherige Lehrveranstaltungen
- ▶ Besonders hilfreich:
 - ▶ Was hat Ihnen gut gefallen?
 - ▶ Wo sehen Sie Verbesserungsbedarf?
- ▶ Zugang: EvaSys-Block auf ihrem Dashboard in Moodle

Evaluationen (evasys)

IVII
SoSe24_08.05.2024_08:00-10:00

Datenbanksysteme_SU_143009_VI_B-MI
WiSe24/25_05.12.2024_14:00-16:00

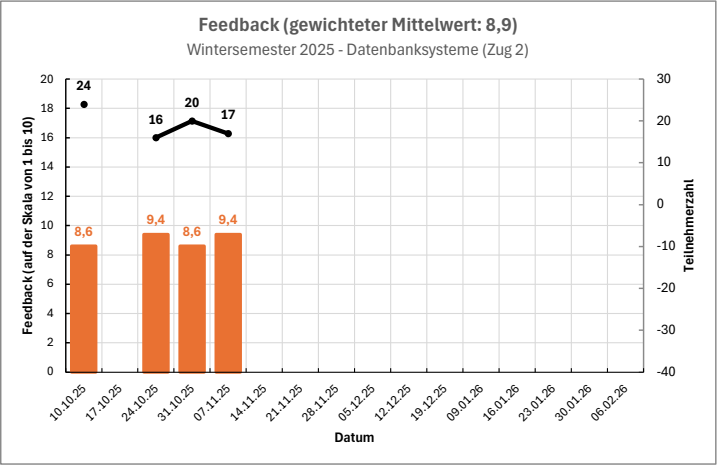
Verteilte Systeme_SU_143151-1_VI_B-MI
WiSe24/25_06.12.2024_08:00-10:00

Verteilte Systeme_SU_143151-2_VI_B-MI
WiSe24/25_05.12.2024_12:00-14:00

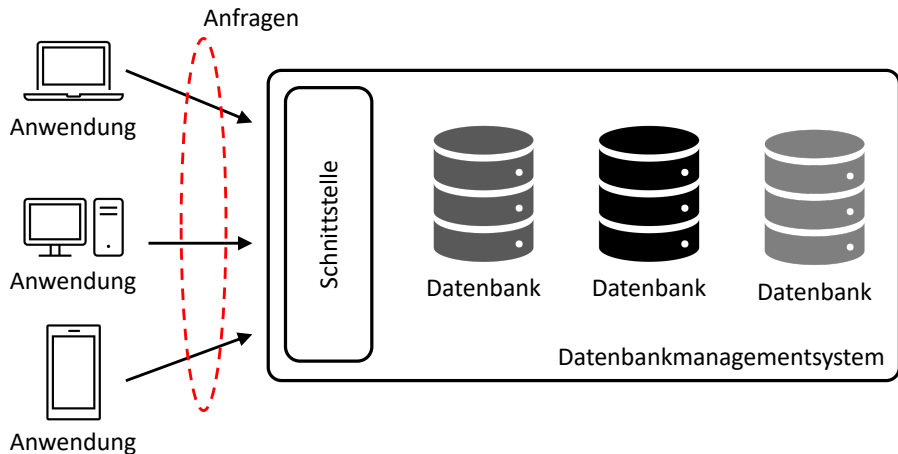
Verteilte Systeme_SU_143151-3_VI_B-MI

All surveys

Feedback



Anfragen mit SQL



Anfragen mit SQL



Quelle: <https://spectrum.ieee.org/top-programming-languages-2025>

SQL

- ▶ SQL-92 [fS92] (auch SQL2, 5x größer im Vergleich zur SQL-89)
- ▶ Lebendige Sprache, zuletzt aktualisiert im Jahr 2023
- ▶ Untersprachen:
 - ▶ SQL-D(ata)Q(uary)L
 - ▶ SQL-DM(anipulation)L
 - ▶ SQL-DD(efinition)L
 - ▶ ...

SQL-QL (Query Language)

```
SELECT <Spalte_1>, ..., <Spalte_k>  
[INTO <Klausel>]  
FROM <tabellenname_1>, ..., <Tabellenname_k>  
[WHERE <Klausel>]  
[GROUP BY <Klausel>]  
[HAVING <Klausel>]  
[UNION <Klausel>]  
[ORDER BY <Klausel>]  
[FOR UPDATE OF (<Spalte_1>, ..., <Spalte_k>)]  
[SAVE TO TEMP <...>];
```

SELECT, WHERE, GROUP BY

- ▶ **SELECT** * **FROM** city;
- ▶ **SELECT** name, district **FROM** city
WHERE name **IN** ('Istanbul', 'Moscow');
- ▶ **SELECT** count(**DISTINCT** name) **FROM** city;
- ▶ **SELECT** countrycode, **sum**(population)
FROM city **GROUP BY** countrycode;
- ▶ **SELECT** district, **count**(*)
FROM city **WHERE** countrycode='CHN'
GROUP BY district **HAVING** **count**(*)>=25;
- ▶ **SELECT** countrycode, **sum**(population)
FROM city **GROUP BY** countrycode
ORDER BY **sum**(population) **DESC LIMIT** 1;

```
SELECT <Spalte_1>, ..., <Spalte_k>  
[INTO <Klausel>]  
FROM <tabellenname_1>, ..., <Tabellenname_k>  
[WHERE <Klausel>]  
[GROUP BY <Klausel>]  
[HAVING <Klausel>]  
[UNION <Klausel>]  
[ORDER BY <Klausel>]  
[FOR UPDATE OF (<Spalte_1>, ..., <Spalte_k>)]  
[SAVE TO TEMP <...>];
```

SELECT, WHERE, GROUP BY

- ▶ **SELECT** * **FROM** city;
- ▶ **SELECT** name, district **FROM** city
WHERE name **IN** ('Istanbul', 'Moscow');
- ▶ **SELECT** count(**DISTINCT** name) **FROM** city;
- ▶ **SELECT** countrycode, sum(population)
FROM city **GROUP BY** countrycode;
- ▶ **SELECT** district, count(*)
FROM city **WHERE** countrycode='CHN'
GROUP BY district **HAVING** count(*)>=25;
- ▶ **SELECT** countrycode, sum(population)
FROM city **GROUP BY** countrycode
ORDER BY sum(population) **DESC LIMIT** 1;

```
SELECT <Spalte_1>, ..., <Spalte_k>  
[INTO <Klausel>]  
FROM <tabellenname_1>, ..., <Tabellenname_k>  
[WHERE <Klausel>]  
[GROUP BY <Klausel>]  
[HAVING <Klausel>]  
[UNION <Klausel>]  
[ORDER BY <Klausel>]  
[FOR UPDATE OF (<Spalte_1>, ..., <Spalte_k>)]  
[SAVE TO TEMP <...>;]
```

SELECT, WHERE, GROUP BY

- ▶ **SELECT** * **FROM** city;
- ▶ **SELECT** name, district **FROM** city
WHERE name **IN** ('Istanbul', 'Moscow');
- ▶ **SELECT** count(DISTINCT name) **FROM** city;
- ▶ **SELECT** countrycode, sum(population)
FROM city **GROUP BY** countrycode;
- ▶ **SELECT** district, count(*)
FROM city **WHERE** countrycode='CHN'
GROUP BY district **HAVING** count(*)>=25;
- ▶ **SELECT** countrycode, sum(population)
FROM city **GROUP BY** countrycode
ORDER BY sum(population) **DESC LIMIT** 1;

```
SELECT <Spalte_1>, ..., <Spalte_k>  
[INTO <Klausel>]  
FROM <tabellenname_1>, ..., <Tabellenname_k>  
[WHERE <Klausel>]  
[GROUP BY <Klausel>]  
[HAVING <Klausel>]  
[UNION <Klausel>]  
[ORDER BY <Klausel>]  
[FOR UPDATE OF (<Spalte_1>, ..., <Spalte_k>)]  
[SAVE TO TEMP <...>];
```

SELECT, WHERE, GROUP BY

- ▶ `SELECT * FROM city;`
- ▶ `SELECT name, district FROM city
WHERE name IN ('Istanbul', 'Moscow');`
- ▶ `SELECT count(DISTINCT name) FROM city;`
- ▶ `SELECT countrycode, sum(population)
FROM city GROUP BY countrycode;`
- ▶ `SELECT district, count(*)
FROM city WHERE countrycode='CHN'
GROUP BY district HAVING count(*)>=25;`
- ▶ `SELECT countrycode, sum(population)
FROM city GROUP BY countrycode
ORDER BY sum(population) DESC LIMIT 1;`

```
SELECT <Spalte_1>, ..., <Spalte_k>  
[INTO <Klausel>]  
FROM <tabellenname_1>, ..., <Tabellenname_k>  
[WHERE <Klausel>]  
[GROUP BY <Klausel>]  
[HAVING <Klausel>]  
[UNION <Klausel>]  
[ORDER BY <Klausel>]  
[FOR UPDATE OF (<Spalte_1>, ..., <Spalte_k>)]  
[SAVE TO TEMP <...>];
```

SELECT, WHERE, GROUP BY

- ▶ `SELECT * FROM city;`
- ▶ `SELECT name, district FROM city
WHERE name IN ('Istanbul', 'Moscow');`
- ▶ `SELECT count(DISTINCT name) FROM city;`
- ▶ `SELECT countrycode, sum(population)
FROM city GROUP BY countrycode;`
- ▶ `SELECT district, count(*)
FROM city WHERE countrycode='CHN'
GROUP BY district HAVING count(*)>=25;`
- ▶ `SELECT countrycode, sum(population)
FROM city GROUP BY countrycode
ORDER BY sum(population) DESC LIMIT 1;`

```
SELECT <Spalte_1>, ..., <Spalte_k>  
[INTO <Klausel>]  
FROM <tabellenname_1>, ..., <Tabellenname_k>  
[WHERE <Klausel>]  
[GROUP BY <Klausel>]  
[HAVING <Klausel>]  
[UNION <Klausel>]  
[ORDER BY <Klausel>]  
[FOR UPDATE OF (<Spalte_1>, ..., <Spalte_k>)]  
[SAVE TO TEMP <...>];
```

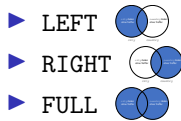
SELECT, WHERE, GROUP BY

- ▶ `SELECT * FROM city;`
- ▶ `SELECT name, district FROM city
WHERE name IN ('Istanbul', 'Moscow');`
- ▶ `SELECT count(DISTINCT name) FROM city;`
- ▶ `SELECT countrycode, sum(population)
FROM city GROUP BY countrycode;`
- ▶ `SELECT district, count(*)
FROM city WHERE countrycode='CHN'
GROUP BY district HAVING count(*)>=25;`
- ▶ `SELECT countrycode, sum(population)
FROM city GROUP BY countrycode
ORDER BY sum(population) DESC LIMIT 1;`

```
SELECT <Spalte_1>, ..., <Spalte_k>  
[INTO <Klausel>]  
FROM <tabellenname_1>, ..., <Tabellenname_k>  
[WHERE <Klausel>]  
[GROUP BY <Klausel>]  
[HAVING <Klausel>]  
[UNION <Klausel>]  
[ORDER BY <Klausel>]  
[FOR UPDATE OF (<Spalte_1>, ..., <Spalte_k>)]  
[SAVE TO TEMP <...>];
```

Wie kann ich mehrerer Tabellen zu einer Ergebnistabelle vereinigen?

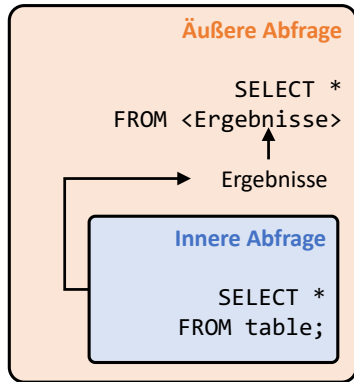
- ▶ CROSS JOIN → Kartesisches Produkt (Kreuzprodukt)
- ▶ INNER JOIN → Verknüpfung von Datensätzen mittels Verbundbedingung
- ▶ OUTER JOIN → wie INNER aber mit Datensätzen, zu denen es keine Entsprechungen der Werte gibt



```
SELECT <Spalte_1>, ..., <Spalte_k>  
[INTO <Klausel>]  
FROM <tabellenname_1>, ..., <Tabellenname_n>  
[WHERE <Klausel>]  
[GROUP BY <Klausel>]  
[HAVING <Klausel>]  
[UNION <Klausel>]  
[ORDER BY <Klausel>]  
[FOR UPDATE OF (<Spalte_1>, ..., <Spalte_n>)]  
[SAVE TO TEMP <...>];
```


Subqueries

- ▶ Ermöglichen schrittweise definition von Abfragen
- ▶ Innere Abfrage liefert Teilergebnisse
- ▶ Äußere Abfrage verarbeitet die Teilergebnisse weiter
- ▶ Dank Ihrer Struktur sind Subqueries einfacher zu lesen



Single-Row-Subquery – Beispiel 1

- Ermittle das Land in Europa mit der höchsten Bevölkerungszahl

Single-Row-Subquery – Beispiel 1

- ▶ Ermittle das Land in Europa mit der höchsten Bevölkerungszahl
- ▶ *--SELECT name, max(population)*
--FROM country;

Data Output Messages Notifications

```
ERROR: column "country.name" must appear in the GROUP BY
clause or be used in an aggregate function
LINE 1: select name, count(population)
              ^
```

SQL state: 42803
Character: 8

Total rows: 239 of 239 Query complete 00:00:00.030 Ln 75,

Single-Row-Subquery – Beispiel 1

- ▶ Ermittle das Land in Europa mit der höchsten Bevölkerungszahl
- ▶ *--SELECT name, max(population)*
--FROM country;
- ▶ **SELECT** name, population
FROM country
ORDER BY 2 DESC LIMIT 1;

Data Output

Messages

Notifications

	name text	population integer
1	China	1277558000

Total rows: 1 of 1

Query complete 00:00:00.045

Single-Row-Subquery – Beispiel 1

- ▶ Ermittle das Land in Europa mit der höchsten Bevölkerungszahl
- ▶ *--SELECT name, max(population)*
--FROM country;
- ▶ **SELECT** name, population
FROM country
ORDER BY 2 **DESC LIMIT** 1;
- ▶ **SELECT** name, population
FROM country
WHERE population = (
 SELECT **max**(population)
 FROM country);

Data Output			Messages	Notifications
        				
	name text		population integer	
1	China		1277558000	

Total rows: 1 of 1 Query complete 00:00:00.045

Subquery – Unterabfrage

- ▶ Subqueries sind universell einsetzbar
- ▶ Jede Abfrage kann eine Unterabfrage sein ...
- ▶ ...Unterabfragen können weitere Unterabfragen enthalten
- ▶ Einschränkungen bei den Unterabfragen:
 - ▶ dürfen nur Bestandteil einer WHERE- oder HAVING-Klausel sein (as per SQL 92)
 - ▶ müssen eine passende Anzahl von Ergebnissen liefern
 - ▶ müssen einen passenden Typ der Ergebnisse liefern

Subquery – Unterabfrage

Single-Row-Subquery

Liefert nur eine Zeile als Ergebnis zurück. Werte können direkt verglichen werden.

Multi-Row-Subquery

Liefert mehrere Zeilen als Ergebnis zurück. Satzoperatoren (IN, ANY, ALL, EXISTS) werden zum Vergleich benutzt.

Single-Row-Subquery – Beispiel 2

- Ermittle alle Länder mit einer überdurchschnittlichen Bevölkerungszahl.

▼	 country
▼	 Columns (15)
	 code
	 name
	 continent
	 region
	 surfacearea
	 indepyear
	 population
	 lifeexpectancy
	 gnp
	 gnpold
	 localname
	 governmentform
	 headofstate
	 capital
	 code2

Single-Row-Subquery – Beispiel 2

- ▶ Ermittle alle Länder mit einer überdurchschnittlichen Bevölkerungszahl.
- ▶ `SELECT name, population/surfacearea`
`FROM country`
`WHERE population/surfacearea > (`
`SELECT AVG(population/surfacearea)`
`FROM country`
`);`

Data Output

Messages

Notifications

+

📄

▼

📄

▼

🗑️

🗄️

⬇️

📈

	name text	?column? double precision
1	Aruba	533.6787564766839
2	Bahrain	889.0489913544668
3	Bangladesh	896.9221794747149
4	Barbados	627.9069767441861
5	Bermuda	1226.4150943396226
6	Gibraltar	4166.666666666667
7	Hong Kong	6298.83200202225

Total rows: 21 of 21

Query complete 00:00:00.039

Single-Row-Subquery – Beispiel 3

- ▶ Wie hoch ist die Bevölkerungszahl der Hauptstadt des am dichtesten besiedelten Landes der Welt?

Data Output		Messages	Notifications
        			
	population integer 		
1	437500		
Total rows: 1 of 1		Query complete 00:00:00.027	

Single-Row-Subquery – Beispiel 3

- ▶ Wie hoch ist die Bevölkerungszahl der Hauptstadt des am dichtesten besiedelten Landes der Welt?
- ▶

```
SELECT population
FROM city
WHERE countrycode = (
    SELECT code
    FROM country
    WHERE population/surfacearea = (
        SELECT MAX(population
                    /surfacearea)
        FROM country
    )
);
```

Data Output		Messages	Notifications
        			
	population integer 		
1	437500		
Total rows: 1 of 1		Query complete 00:00:00.027	

Auf ins Ausland

Studium, Praktikum oder Abschlussarbeit –
in Europa & weltweit

- ✓ Neue Perspektiven, neue Leute, neue Chancen!
- ✓ Top für Lebenslauf & Lebenserfahrung
- ✓ Beratung und Unterstützung

Infos &
Beratung auf
unserer
Webseite



www.bht-berlin.de/international

Die passende
Partnerhoch-
schule finden &
Erfahrungs-
berichte
Studium



[Mobility Online](#)

Tipps, Infos &
Erfahrungs-
berichte
Praktikum im
Moodle-Kurse



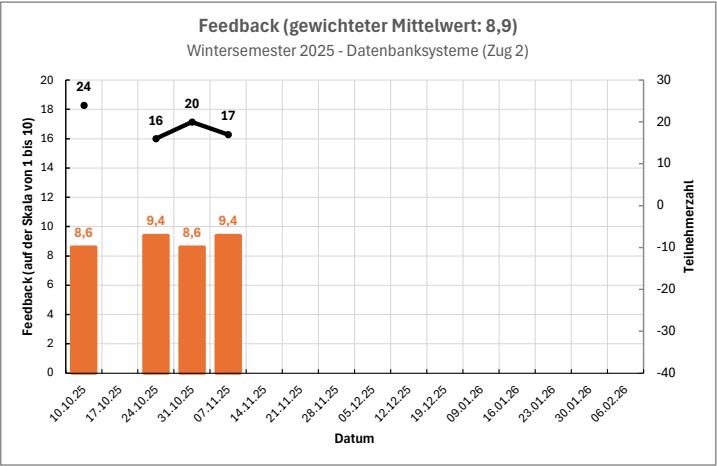
Einschreibeschlüssel: **refint**

BHT
Berliner Hochschule
für Technik

Studiere Zukunft



Feedback



Lehreveranstaltungsevaluation

Durchschnitt: 1,5 – schlechteste Bewertung: 1,9

- ▶ [...] gutes Zeitmanagement
- ▶ [...] die meisten Studierenden sich aktiv beteiligen
- ▶ [...] den Stellenwert des Lehrinhalts für das weitere Studium klar machen

Wo sehen Sie Verbesserungsbedarf?

- ▶ mehr Übungsaufgaben bezüglich SQL queries
- ▶ Ich bin ein Fan von Pünktlichkeit
- ▶ langsamer/mehr erklären (Terminal / pgAdmin)

Multi-Row-Subquery

- Finde Namen aller Hauptstädte

Data Output		Messages	Notifications
        			
	name text 		
1	Kabul		
2	Amsterdam		
3	Willemstad		
4	Tirana		
5	Alger		
6	Forstegg		
Total rows: 232 of 232		Query complete 00:00:00.032	

Multi-Row-Subquery

- ▶ Finde Namen aller Hauptstädte
- ▶

```
SELECT ci.name
FROM city AS ci, country AS co
WHERE ci.id=co.capital;
```
- ▶

```
SELECT name
FROM city
WHERE id IN (
    SELECT capital
    FROM country );
```

Data Output		Messages	Notifications
	name		
	text		
1	Kabul		
2	Amsterdam		
3	Willemstad		
4	Tirana		
5	Alger		
6	... (truncated)		
Total rows: 232 of 232		Query complete 00:00:00.032	

Multi-Row-Subquery – (NOT) IN, ANY, ALL, EXISTS – 1/3

```
► SELECT name  
FROM city  
WHERE id IN (  
    SELECT capital  
    FROM country );
```

Data Output		Messages	Notifications
        			
	name text 		
1	Kabul		
2	Amsterdam		
3	Willemstad		
4	Tirana		
5	Alger		
6	Egypte		
Total rows: 232 of 232		Query complete 00:00:00.032	

Multi-Row-Subquery – (NOT) IN, ANY, ALL, EXISTS – 1/3

- ▶ `SELECT name`
`FROM city`
`WHERE id IN (`
 `SELECT capital`
 `FROM country );`
- ▶ `SELECT name`
`FROM city`
`WHERE id = ANY (`
 `SELECT capital`
 `FROM country );`

Data Output		Messages	Notifications
        			
	name text 		
1	Kabul		
2	Amsterdam		
3	Willemstad		
4	Tirana		
5	Alger		
6	Egypten		
Total rows: 232 of 232		Query complete 00:00:00.032	

Multi-Row-Subquery – (NOT) IN, ANY, ALL, EXISTS – 2/3

► **SELECT** name, population
FROM city
WHERE population <
 ANY (**VALUES** (100), (200));

Data Output

Messages

Notifications

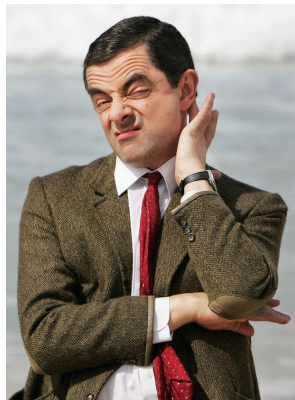
	name text	population integer
1	West Island	167
2	Adamstown	42

Total rows: 2 of 2

Query complete 00:00:00.027

Multi-Row-Subquery – (NOT) IN, ANY, ALL, EXISTS – 2/3

- ▶ `SELECT name, population`
`FROM city`
`WHERE population <`
`ANY ( VALUES (100), (200));`
- ▶ `SELECT name, population`
`FROM city`
`WHERE population <`
`ALL ( VALUES (100), (200));`



Multi-Row-Subquery – (NOT) IN, ANY, ALL, EXISTS – 2/3

- ▶ `SELECT name, population`
`FROM city`
`WHERE population <`
`ANY ( VALUES (100), (200));`
- ▶ `SELECT name, population`
`FROM city`
`WHERE population <`
`ALL ( VALUES (100), (200));`

Data Output

Messages

Notifications

≡+

▼

▼

	name text	population integer
1	Adamstown	42

Total rows: 1 of 1

Query complete 00:00:00.028

Multi-Row-Subquery – (NOT) IN, ANY, ALL, EXISTS – 3/3

```
► SELECT name, population
   FROM city WHERE
   countrycode='DEU'
  AND EXISTS (
    SELECT * FROM city
    WHERE name = 'Berlin')
```

Data Output

Messages

Notifications



















	name text	population integer
1	Berlin	3386667
2	Hamburg	1704735
3	Munich [München]	1194560
4	Köln	962507
5	Frankfurt am Main	643821
6	Essen	589515

Total rows: 93 of 93

Query complete 00:00:00.028

Multi-Row-Subquery – (NOT) IN, ANY, ALL, EXISTS – 3/3

- ▶ `SELECT name, population`
`FROM city WHERE`
`countrycode='DEU'`
`AND EXISTS (`
 `SELECT * FROM city`
 `WHERE name = 'Berlin')`
- ▶ `SELECT name, population`
`FROM city WHERE`
`countrycode='DEU'`
`AND EXISTS (`
 `SELECT * FROM city`
 `WHERE name = 'Wedding')`

Data Output		Messages	Notifications
			
name text		population integer	
Total rows: 0 of 0		Query complete 00:00:00.028	

Correlated Subquery

- ▶ Eine gemeinsame Spalte (Korrelationsvariable) zwischen Haupt- und Unterabfrage
- ▶ Subquery wird für jede Zeile der Haupttabelle neu ausgewertet ...
- ▶ ...wodurch die gesamte Abfrage auch langsam werden kann

```
▶ SELECT ci.name
FROM city ci, country co
WHERE ci.id = co.capital;
-- 00:00:00.029
```


Correlated Subquery

- ▶ Eine gemeinsame Spalte (Korrelationsvariable) zwischen Haupt- und Unterabfrage
- ▶ Subquery wird für jede Zeile der Haupttabelle neu ausgewertet ...
- ▶ ...wodurch die gesamte Abfrage auch langsam werden kann

```

▶ SELECT ci.name
   FROM city ci, country co
  WHERE ci.id = co.capital;
-- 00:00:00.029

▶ SELECT ci.name FROM city ci
  WHERE ci.id IN ( SELECT capital
                  FROM country );
-- 00:00:00.041
    
```

Correlated Subquery

- ▶ Eine gemeinsame Spalte (Korrelationsvariable) zwischen Haupt- und Unterabfrage
- ▶ Subquery wird für jede Zeile der Haupttabelle neu ausgewertet ...
- ▶ ...wodurch die gesamte Abfrage auch langsam werden kann

- ▶

```
SELECT ci.name
FROM city ci, country co
WHERE ci.id = co.capital;
-- 00:00:00.029
```
- ▶

```
SELECT ci.name FROM city ci
WHERE ci.id IN ( SELECT capital
FROM country );
-- 00:00:00.041
```
- ▶

```
SELECT ci.name FROM city ci
WHERE ci.id IN ( SELECT capital
FROM country
WHERE capital=ci.id );
-- 00:00:00.153
```

Correlated-Subquery – Beispiel 1

- Wie ist das Verhältnis von in Städten lebenden Menschen zur Gesamtbevölkerung pro Land?



Correlated-Subquery – Beispiel 1

- ▶ Wie ist das Verhältnis von in Städten lebenden Menschen zur Gesamtbevölkerung pro Land?

```
▶ SELECT co.name,  
    ( SELECT SUM(population)  
    FROM city ci  
    WHERE ci.countrycode=co.code  
    GROUP BY ci.countrycode )  
    / co.population::float  
FROM country co  
WHERE co.population > 0  
ORDER BY 2 DESC;
```

Data Output

Messages

Notifications

≡

+

▼

▼

	name text		citypopulationperc double precision	
1	Singapore		1.1263619287917017	
2	Cocos (Keeling) Islands		1.1166666666666667	
3	Gibraltar			1.081
4	Macao		0.9249471458773785	
5	Pitcairn			0.84
6	South Korea		0.8225482002010717	

Total rows: 232 of 232

Query complete 00:00:00.078

Correlated-Subquery – Beispiel 1

- Wie ist das Verhältnis von in Städten lebenden Menschen zur Gesamtbevölkerung pro Land?
- ```

SELECT co.name,
 (SELECT SUM(population)
 FROM city ci
 WHERE ci.countrycode=co.code
 GROUP BY ci.countrycode)
 / co.population::float
FROM country co
WHERE co.population > 0
ORDER BY 2 DESC;

```

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

📄

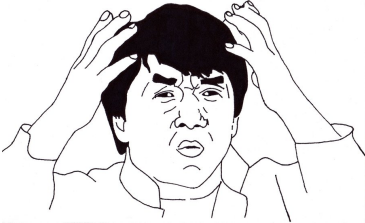
⬇️

📈

|   | name<br>text            | citypopulationperc<br>double precision |
|---|-------------------------|----------------------------------------|
| 1 | Singapore               | 1.1263619287917017                     |
| 2 | Cocos (Keeling) Islands | 1.1166666666666667                     |
| 3 | Gibraltar               | 1.081                                  |
| 4 | Macao                   | 0.9249471458773785                     |
| 5 | Pitcairn                | 0.84                                   |
| 6 | South Korea             | 0.82354920202018717                    |

Total rows: 232 of 232

Query complete 00:00:00.078



## Correlated-Subquery – Beispiel 2

- ▶ In welchen Ländern leben mehr Menschen in Städten als im gesamten Land?

## Correlated-Subquery – Beispiel 2

- In welchen Ländern leben mehr Menschen in Städten als im gesamten Land?

```
► SELECT ci.countrycode, ci.name,
 ci.population,
 (SELECT population FROM country
 WHERE code=ci.countrycode)
FROM city ci
WHERE ci.countrycode IN
 (SELECT code FROM country co
 WHERE co.population <
 (SELECT sum(population)
 FROM city
 WHERE countrycode=co.code
 GROUP BY countrycode));
```

Data Output

Messages

Notifications

countrycode  
character

name  
text

population  
integer

population  
integer

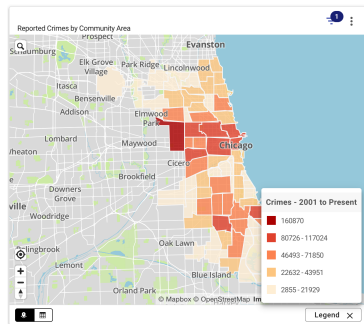
|   |     |              |         |         |
|---|-----|--------------|---------|---------|
| 1 | GIB | Gibraltar    | 27025   | 25000   |
| 2 | CCK | Bantam       | 503     | 600     |
| 3 | CCK | West Isla... | 167     | 600     |
| 4 | SGP | Singapore    | 4017733 | 3567000 |

Total rows: 4 of 4

Query complete 00:00:00.087

# Chicago Crimes Datensatz

- ▶ Informationen zu allen gemeldeten Straftaten in Chicago seit dem Jahr 2001
- ▶ Jede Straftat ist u.a. mit Koordinaten (latitude und longitude), dem Polizeibezirk (beat und district) und dem genauen Datum/Uhrzeit (date) versehen
- ▶ Die Daten sind in Haupt- (primary\_type) und Unterkategorien (description) von Delikten gegliedert

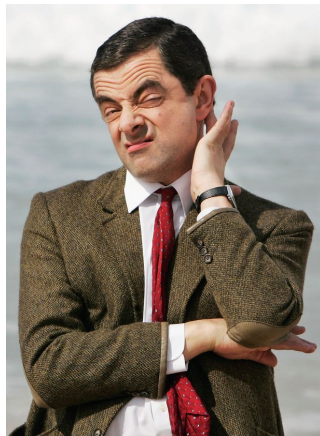


[https://data.cityofchicago.org/  
Public-Safety/  
Crimes-2001-to-Present/  
ijzp-q8t2/about\\_data](https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2/about_data)



## Correlated-Subquery – Beispiel 3

- ▶ Welche Hauptarten von Straftaten haben den höchsten Prozentsatz an Verhaftungen?



# Correlated-Subquery – Beispiel 3

► Welche Hauptarten von Straftaten haben den höchsten Prozentsatz an Verhaftungen?

```
SELECT primary_type,
 100*(count(*)::float / (count(*) + (
SELECT count(*)
FROM crimes c2
WHERE arrest = false
GROUP BY c2.primary_type
HAVING c2.primary_type
 = c1.primary_type
))) as arrest_perc
FROM crimes c1
WHERE arrest = true
GROUP BY primary_type
ORDER BY 2 DESC;
```

Data OutputMessagesNotifications

|    | primary_type<br>character varying (255) | arrest_perc<br>double precision |
|----|-----------------------------------------|---------------------------------|
| 1  | DOMESTIC VIOLENCE                       | [null]                          |
| 2  | PROSTITUTION                            | 99.5794545739661                |
| 3  | NARCOTICS                               | 99.35857727515791               |
| 4  | GAMBLING                                | 99.27610462336953               |
| 5  | LIQUOR LAW VIOLATION                    | 99.09600791817881               |
| 6  | PUBLIC INDECENCY                        | 98.52216748768473               |
| 7  | CONCEALED CARRY LICE...                 | 96.3917525773196                |
| 8  | INTERFERENCE WITH PU...                 | 91.70089958920494               |
| 9  | OBSCENITY                               | 75.93643586833144               |
| 10 | WEAPONS VIOLATION                       | 72.47821410726534               |
| 11 | CRIMINAL TRESPASS                       | 68.88144038047787               |
| 12 | OTHER NARCOTIC VIOLA...                 | 67.97385620915033               |
| 13 | PUBLIC PEACE VIOLATION                  | 62.671117908165165              |
| 14 | HOMICIDE                                | 47.34998136414462               |
| 15 | NON-CRIMINAL (SUBJEC...                 | 33.33333333333333               |
| 16 | SEX OFFENSE                             | 26.12011147520902               |
| 17 | BATTERY                                 | 21.76293667504968               |

# Common Table Expressions

- ▶ Common Table Expressions (CTEs) → Definition von temporären benannten Ergebnismengen innerhalb einer Abfrage
- ▶ Mit WITH können komplexe Abfragen in kleinere, lesbare Teilabfragen zerlegt werden
- ▶ CTEs sind nur während der Ausführung der Hauptabfrage gültig
- ▶ Warum sollte ich CTEs benutzen?
  - ▶ CTEs verbessern die Wartbarkeit des Codes
  - ▶ Materialisierung – CTEs werden nur einmal ausgeführt, auch wenn sie mehrmals verwendet werden

# CTE – Beispiel 1

```
SELECT co.name,
 (SELECT SUM(population)
 FROM city ci
 WHERE ci.countrycode=co.code
 GROUP BY ci.countrycode)
 / co.population::float
FROM country co
WHERE co.population > 0
ORDER BY 2 DESC;
```

```
WITH cityPop AS (
 SELECT countrycode ,
 SUM(population) AS sumPop
 FROM city
 GROUP BY countrycode
)

SELECT co.name,
 p1.sumPop / co.population::float
FROM country co, cityPop p1
WHERE co.code = p1.countrycode
 AND co.population > 0
ORDER BY 2 DESC;
```

## CTE – Beispiel 2

```
SELECT co.name,
 co.population,
 SUM(ci.population)
FROM country AS co
INNER JOIN city AS ci
 ON co.code = ci.countrycode
WHERE co.continent = 'Europe'
GROUP BY co.name, co.population
HAVING co.population
 <= SUM(ci.population);
```

```
WITH EUCountryP AS (
 SELECT name, code,
 population as p
 FROM country
 WHERE continent = 'Europe'
) , CityP AS (
 SELECT countrycode as code,
 sum(population) as p
 FROM city
 GROUP BY code
)
SELECT co.name, co.p, ci.p
FROM EUCountryP co, CityP ci
WHERE co.code = ci.code
AND co.p <= ci.p
```

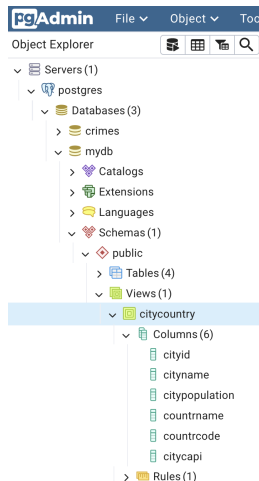
# Views in PostgreSQL

- ▶ Abstraktion komplexer SQL-Abfragen in eine benannte logische Sicht
- ▶ Vereinfachung der Nutzung wiederkehrender Abfrage-Logik
- ▶ Klarere Trennung zwischen Datenmodell und Anwendungslogik
- ▶ Reduktion von Redundanz und Fehleranfälligkeit bei komplexen Abfragen
- ▶ Unterstützung konsistenter Schnittstellen trotz Änderungen im Schema

# Views – Beispiel 1

- ▶ Sicht mit Städten und Ländern
- ▶ 

```
CREATE VIEW cityCountry AS
SELECT
 c.id AS cityId,
 c.Name AS cityName,
 c.Population AS cityPopulation,
 co.Name AS countrName,
 co.Code AS countrCode,
 (select CASE WHEN
 co.capital = c.ID
 THEN true ELSE false END)
 as cityCapi
FROM city c
JOIN country co
ON c.CountryCode = co.Code;
```



# Views – Beispiel 2

- ▶ Sicht mit Städten und Ländern
- ▶ `CREATE VIEW cityCountry`
- ▶ `SELECT countrName, cityName`  
`FROM cityCountry`  
`WHERE cityCapi = 'T'`

Data OutputMessagesNotifications

≡+

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

|                 | <div>countrname</div> <div>text</div> <div>🔒</div> | <div>cityname</div> <div>text</div> <div>🔒</div> |
|-----------------|----------------------------------------------------|--------------------------------------------------|
| 1               | Afghanistan                                        | Kabul                                            |
| 2               | Netherlands                                        | Amsterdam                                        |
| 3               | Netherlands Antilles                               | Willemstad                                       |
| 4               | Albania                                            | Tirana                                           |
| 5               | Algeria                                            | Alger                                            |
| 6               | American Samoa                                     | Fagatogo                                         |
| 7               | Andorra                                            | Andorra la Vella                                 |
| Total rows: 232 |                                                    | Query complete 00:00:00.072                      |



# Materialisierte Views

- ▶ Physische Speicherung des Ergebnisssets einer Abfrage
- ▶ Beschleunigung komplexer Analysen durch vorgerechnete Daten
- ▶ Explizite Aktualisierung per `REFRESH MATERIALIZED VIEW`
- ▶ Vorteil bei großen Datenmengen und seltenen Änderungen
- ▶ Unterschied zu normalen Views: keine Live-Berechnung bei jedem Zugriff
- ▶ Normale Views bleiben stets aktuell, sind aber rechenintensiver bei teuren Abfragen

# Materialisierte Views – Beispiel 1

- ▶ Sicht mit verbrechen  
Kennzahlen pro Jahr
- ▶ 

```
CREATE VIEW crimeAggr AS
SELECT year, primary_type, count(*)
FROM crimes
GROUP BY year, primary_type

SELECT * FROM crimeAggr;
```

| Data Output Messages Notifications                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                 |                                         |                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|-----------------------------------------|-----------------|
| <div> SQL <span>St</span></div> |                 |                                         |                 |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | year<br>integer | primary_type<br>character varying (255) | count<br>bigint |
| 1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 2001            | ARSON                                   | 1011            |
| 2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 2001            | ASSAULT                                 | 31386           |
| 3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 2001            | BATTERY                                 | 93467           |
| 4                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 2001            | BURGLARY                                | 26014           |
| 5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 2001            | CONCEALED CARRY LICENSE VIOLATION       | 1               |
| 6                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 2001            | CRIM SEXUAL ASSAULT                     | 1776            |
| 7                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 2001            | CRIMINAL DAMAGE                         | 55863           |
| Total rows: 751                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                 | Query complete 00:00:01.202             |                 |

## Materialisierte Views – Beispiel 2

- ▶ Sicht mit verbrechen Kennzahlen pro Jahr
- ▶ 

```
CREATE MATERIALIZED VIEW crimeAggrM AS
SELECT year, primary_type, count(*)
FROM crimes
GROUP BY year, primary_type

SELECT * FROM crimeAggrM;
```

Data Output

Messages

Notifications

SQL

|                 | year<br>integer | primary_type<br>character varying (255) | count<br>bigint |
|-----------------|-----------------|-----------------------------------------|-----------------|
| 1               | 2001            | ARSON                                   | 1011            |
| 2               | 2001            | ASSAULT                                 | 31386           |
| 3               | 2001            | BATTERY                                 | 93467           |
| 4               | 2001            | BURGLARY                                | 26014           |
| 5               | 2001            | CONCEALED CARRY LICENSE VIOLATION       | 1               |
| 6               | 2001            | CRIM SEXUAL ASSAULT                     | 1776            |
| 7               | 2001            | CRIMINAL DAMAGE                         | 55865           |
| Total rows: 751 |                 | Query complete 00:00:00.045             |                 |

## Materialisierte Views – Beispiel 2

- ▶ Sicht mit verbrechen Kennzahlen pro Jahr
- ▶ 

```
CREATE MATERIALIZED VIEW crimeAggrM AS
SELECT year, primary_type, count(*)
FROM crimes
GROUP BY year, primary_type

SELECT * FROM crimeAggrM;
```
- ▶ 

```
REFRESH MATERIALIZED VIEW crimeAggrM;
```

Data Output Messages Notifications

REFRESH MATERIALIZED VIEW

Query returned successfully in 1 secs 324 msec.

Total rows: Query complete 00:00:01.324

# Satz-Operatoren – UNION

- ▶ Zusammenfassung von Ergebnissen von mehreren Abfragen

# Satz-Operatoren – UNION

- ▶ Zusammenfassung von Ergebnissen von mehreren Abfragen
- ▶ 

```
SELECT countrycode, name, population, 1
FROM city WHERE countrycode='NOR'
UNION
SELECT code, name, population, 0
FROM country WHERE code='NOR'
ORDER BY 4
```

# Satz-Operatoren – UNION

- ▶ Zusammenfassung von Ergebnissen von mehreren Abfragen
- ▶ 

```
SELECT countrycode, name, population, 1
FROM city WHERE countrycode='NOR'
UNION
SELECT code, name, population, 0
FROM country WHERE code='NOR'
ORDER BY 4
```

| Data Output        |                          |              |                       |                     | Messages                    | Notifications |
|--------------------|--------------------------|--------------|-----------------------|---------------------|-----------------------------|---------------|
|                    | countrycode<br>character | name<br>text | population<br>integer | ?column?<br>integer |                             |               |
| 1                  | NOR                      | Stavanger    | 108848                | 1                   |                             |               |
| 2                  | NOR                      | Bærum        | 101340                | 1                   |                             |               |
| 3                  | NOR                      | Trondheim    | 150166                | 1                   |                             |               |
| 4                  | NOR                      | Norway       | 4478500               | 0                   |                             |               |
| 5                  | NOR                      | Bergen       | 230948                | 1                   |                             |               |
| 6                  | NOR                      | Oslo         | 508726                | 1                   |                             |               |
| Total rows: 6 of 6 |                          |              |                       |                     | Query complete 00:00:00.031 |               |

# Satz-Operatoren – INTERSECT

- Sowohl als auch...



# Satz-Operatoren – INTERSECT

► Sowohl als auch...

```
► SELECT name
 FROM city
 WHERE id IN
 (
 SELECT capital
 FROM country
 WHERE lifeexpectancy>80
 INTERSECT
 SELECT capital
 FROM country
 WHERE population > 1000000)
```

# Satz-Operatoren – INTERSECT

► Sowohl als auch...

```
► SELECT name
FROM city
WHERE id IN
(
 SELECT capital
 FROM country
 WHERE lifeexpectancy>80
 INTERSECT
 SELECT capital
 FROM country
 WHERE population > 1000000)
```

Data Output

Messages

Notifications

≡

+

|   | name      |  |
|---|-----------|--|
|   | text      |  |
| 1 | Singapore |  |
| 2 | Tokyo     |  |

Total rows: 2 of 2

Query complete 00:00:00.031

# Satz-Operatoren – EXCEPT

- Mengen subtrahieren

# Satz-Operatoren – EXCEPT

- ▶ Mengen subtrahieren
- ▶ `SELECT` name, population  
`FROM` city  
`EXCEPT`  
`SELECT` name, population  
`FROM` city  
`WHERE` population >100

# Satz-Operatoren – EXCEPT

- Mengen subtrahieren
- `SELECT` name, population  
`FROM` city  
`EXCEPT`  
`SELECT` name, population  
`FROM` city  
`WHERE` population >100

| Data Output                                                                                                                           |              | Messages                    | Notifications |
|---------------------------------------------------------------------------------------------------------------------------------------|--------------|-----------------------------|---------------|
| <div> <div>≡+</div> <div>📄</div> <div>▼</div> <div>📋</div> <div>▼</div> <div>🗑️</div> <div>🗄️</div> <div>⬇️</div> <div>📈</div> </div> |              |                             |               |
|                                                                                                                                       | name<br>text | population<br>integer       |               |
| 1                                                                                                                                     | Adamstown    | 42                          |               |
| Total rows: 1 of 1                                                                                                                    |              | Query complete 00:00:00.065 |               |

# Satz-Operatoren – EXCEPT

- ▶ Mengen subtrahieren
- ▶ `SELECT` name, population  
`FROM` city  
`EXCEPT`  
`SELECT` name, population  
`FROM` city  
`WHERE` population >100
- ▶ `--SELECT` name, population  
`--FROM` city  
`--WHERE` population <=100

| Data Output                                                                         |                                                                                     | Messages                                                                            | Notifications                                                                       |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|   |  |  |  |
|  |  |  |  |
|  |                                                                                     |                                                                                     |                                                                                     |
|                                                                                     | name<br>text                                                                        | population<br>integer                                                               |                                                                                     |
| 1                                                                                   | Adamstown                                                                           | 42                                                                                  |                                                                                     |
| Total rows: 1 of 1    Query complete 00:00:00.065                                   |                                                                                     |                                                                                     |                                                                                     |

# Zusammenfassung

- ▶ Sub-Queries
  - ▶ Single-Row-Subquery
  - ▶ Multi-Row-Subquery
    - ▶ (NOT) IN
    - ▶ ANY
    - ▶ ALL
    - ▶ EXISTS
  - ▶ Correlated Subquery
- ▶ Satz-Operatoren
  - ▶ UNION
  - ▶ INTERSECT
  - ▶ EXCEPT

# Umfrage!



#Z2DB06

<https://sli.do>



# Referenzen

[fS92] International Organization for Standardization. Database language SQL. online, July 1992.